

Service creation tools for speech interactive services

A Chester, R Hanes and J Cheshire

This paper discusses service creation tools and the benefits which these tools can deliver. The BT VISAGE (voice interactive services application generation environment) service creation tool is used as a specific example and is described in some detail. VISAGE is currently used within BT to generate advanced network-based services, an example of this being the network-based call-answering service 'CallMinder' which runs on the interactive speech applications platform (ISAP).

1. Introduction

Interactive speech technology is being increasingly used in many areas of the telecommunications industry and elsewhere. The type and complexity of interactive speech services varies considerably, from simple voice announcements services (such as Callstream type services) to highly complex and technically advanced systems such as automated directory enquiries and CallMinder™ [1]. The services are developed by a range of people, including skilled programmers, trained developers and end customers wishing to make small modifications to their voice-announcement services. The applications or services developed target a variety of hardware platforms and use different types and combinations of voice technology, e.g. speech recognition, TouchTone¹ recognition and voice output.

As the technology matures and service providers become more confident with the use of speech interactive services, demand for the rapid deployment of new high-quality and low-cost services increases. Service creation tools are seen as the mechanism to enable this.

Service creation tools (SCTs) are software tools that enable a service developer to create custom applications rapidly. Additionally they may provide system management functions, e.g. management information and system configuration. These tools may be supplied as a suite of software with a common user interface for all user functions.

2. Service creation tool benefits

The key benefits which arise from the use of service creation tools are:

- separation of skill sets — specialist service designers can concentrate on the important aspects of service design without the encumbrance of working with conventional programming languages,

- reduction of application development time — SCTs permit interactive speech services to be implemented quickly and inexpensively in order to respond to a growing market-place where time to market is often critical,
- establishing standards — emerging standards with regards to dialogue interaction can be incorporated into the tool, allowing services to have a consistent 'look and feel',
- an aid to understanding the service — because services can be represented in a graphical form, they can be appreciated by a much broader range of people than would normally be possible with services described purely in textual format.

In order to achieve the above, it is important that the tool-set is easy to use and consistent across the different aspects of service creation addressed. In addition, to be fully effective, the tool-set should cover all major aspects of service development and system management.

The features noted above reflect the underlying benefits that tools offer, i.e. making the developer's job easier, allowing high-quality services to be created quickly and at reduced cost.

3. Service creation tool functionality

3.1 Tools for service development

The types of tools used for service development may be grouped into the following three broad categories.

¹ TouchTone™, known technically as dual tone multiple frequency (DTMF), are the tones sent when the keys are pressed on a tone dialling telephone.

- Point and click icon-based tools which use a graphical user interface — these flowcharting tools, based upon icons, fulfil a threefold purpose. They assist customers in thinking through the sequence of steps or actions which are required to achieve the particular objective, secondly they may be used as a basis for communication within the service project team, and thirdly they form the basis for generating the service code itself for downloading on to the target speech applications platform.
- Menu-based tools — these are based on index card/form filing menus and use tree structures to define the service. Each card/form within the tree consists of voice prompts and actions which are used to define what happens and what the caller may do next.
- Tools which use a scripting language — applications are written as high-level macro commands that are either interpreted or compiled. These high-level commands cater for a wide range of user interaction and permit speech interactive functionality to be written swiftly, to construct complex programs compared with conventional approaches using, for example, C code.

Although the types of tools identified above can all be used to assist with the requirements capture and service creation process, their usefulness overall is often linked to the availability of other features which can assist the service designer and developer. For example, such tools may also provide help with testing, documentation and recoding of the messages to be used in a service, together with providing an environment for undertaking simulations of the service, as follows.

Service validation — validation routines are frequently used to check the consistency of a dialogue, ensuring that there are no unconnected paths or incompletely specified actions which might disrupt call flow. In addition, the messages (prompts) which need to be recorded can be defined as the design progresses. These facilities cannot, however, ensure that a service will be user friendly and will successfully achieve the original design objectives. These criteria depend on the approach to testing and enhancing effectiveness of a new design, coupled with the experience of the service designer.

Documentation — the client for a particular service will have an understanding of the 'feel' they wish to achieve for the service under development, while the developer knows how to specify the service using the tool-set. To help the two parties communicate at a common level, good service documentation is essential. Such documentation can depict the flow of a service and list the text for the messages or prompts that will ultimately be played out to the service user. Within this framework, the two parties can then participate in the design process from a basis of common knowledge and understanding. As the ebb and flow of ideas

may cause the design to change rapidly, tools that can automatically generate the documentation for the evolving service take on what would otherwise be a laborious task.

Message preparation — the quality of voice recordings or 'prompts' used within a service has a significant influence on the user perception of a service. As such, good recording and editing tools that allow the cutting and pasting of speech, removal of unwanted silence and the encoding of the speech messages at different bit rates are essential in producing a quality service.

Service simulation — service simulation tools provide a means of simulating a service within the confines of a local development platform, e.g. a PC, without the need to generate and download the software on to a target speech applications platform. By using simulations, the effectiveness of the service can be assessed quickly and easily and any necessary modifications can be made. This allows service developers to experiment with and improve the service dialogues until the desired effect has been achieved.

3.2 Tools for service and platform management

Although system and service management does not generally affect the design of a service dialogue and therefore is not usually visible to the customer, in-life management forms a key part of the development of a new service. A variety of tools can be used to assist with the dimensioning and management of the various service and platform resources, to provide management information and assist maintenance. For example, flood-test call-simulation tools allow a service to be stress tested by simulating the likely in-service conditions. A variety of loads (i.e. the number of calls received by the service under test) can be simulated, in such a way that the optimum allocation of system resources can be applied to minimise the timing or system performance differences under the differing load patterns.

Management information and maintenance tools are often associated with the target platform rather than the process of service development. They are essential for the collection, processing and presentation of statistics, software back-up and alarm management. This is especially useful when used in conjunction with database tools to allow local and remote database management, user access levels, protecting and recovering data, etc.

System configuration, management and maintenance tools permit local or remote terminal access of functionality that includes, for example:

- line assignment,
- busy out line for maintenance,
- error thresholds,
- passwords,
- host system status.

Speech recognition is still relatively immature and currently a number of speech services request information from callers which later has to be transcribed by human operator, e.g. names and addresses of callers. Call-transcription tools can assist the operators step through the various speech files, allowing the important information to be retrieved quickly and easily. For services which use speech recognition, these call-transcription tools have evolved into dialogue replay tools which can use stored event logs and recordings of any utterances spoken by the user to recreate a specific interaction with the service. This helps greatly with the assessment of the usability of the service and hence is key to improving the service as operational experience is gained.

From the above it can be seen that a wide variety of tools can (and should) be used to assist and support the specification, development and management of speech-based services. In many cases, the tools that are available perform a specific and specialised role. This can lead to a fragmented approach to service design and development, and create difficulties in moving from one development phase to another. This is not entirely unexpected, as the deployment of this new technology to meet the market needs is currently running ahead of the agreement of standards which would be needed for a truly open system.

4. VISAGE

VISAGE (voice interactive service application generation environment) was developed by BT as a means of rapidly developing new interactive speech-based services for a variety of speech platforms. It offers an integrated set of tools which support the rapid creation, testing, simulation, code generation and documentation of speech interactive services.

These tools are helping BT to build high-quality speech-based applications quickly and cost effectively and to promote a consistent look and feel to BT's speech-based services [2]. In addition, all service documentation produced by VISAGE is of a standard format. The use of VISAGE has been demonstrated to reduce service development time very significantly. Using the tool-set, a basic service can be taken from an idea through to a flowchart, message recording, service simulation and then have the code produced for it, in a matter of hours or days rather than months. This facilitates the rapid implementation of technologically advanced solutions within BT's network.

VISAGE allows application developers to concentrate on the most important aspect of application development, namely meeting customer requirements at lowest cost as quickly as possible, without having to be overly concerned with the low-level mechanisms of the application. Compared to conventional programming using expert computer programmers, the use of this high-level programming tool permits development of services with a more appropriate skill-level requirement. The tool also has on-line, context-sensitive help to assist the service developer.

Service simulation enables the service developer to test out services rapidly without having to compile the service into an executable code for downloading on to the target speech platform. When the developer and customer are satisfied with the service as simulated, VISAGE can produce C++ source code in a consistent and standard manner for use on the target speech application platform.

VISAGE offers an intuitive, user friendly, graphical interface which supports the following tool suite:

- a flowcharting tool, based on the concept of generic application building blocks (see section 5.2),
- a message recording tool and editing tools (see section 5.3),
- a service simulation tool for true service simulation using a telephony interface, a VDU screen-only mode and 'Wizard of Oz' service simulation capability where speech recognition is performed by the service developer (see section 5.4),
- a code generator which produces C++ code for execution on the target speech application platform (see section 5.5).

VISAGE is designed around the concept of code reuse of a family of generic building blocks that cover telephony functions, dialogue interaction, control flow and miscellaneous functions. The dialogue blocks use standard, proven caller interactions and have been tested extensively using human factors trials [2].

The tools in their current form may be used to construct both simple and more complex services, whose major functional content is dialogue interactions. Database, host interaction, signalling and billing functions may be incorporated, but at present the service developer is required to write these function in a conventional programming language.

4.1 Visage architecture

The architecture of the visage tool-set reflects the underlying design approach illustrated in Fig 1. The initial

ideas and service design are captured and refined using the flowcharter tool. From this graphical representation of the service, an intermediate service script is generated. In addition, details of the messages to be recorded are generated and are used by the message-recording tool to assist with the message recording and preparation. Using the recorded messages and service script, it is now possible to simulate the service using the service simulation tool. Two modes of simulation are possible, a screen-based approach whereby prompts are displayed on a computer screen and responses typed in accordingly, and speech simulation where a person interacts with the system using speech and TouchTone key presses. Information gained from the simulation is then used to refine the service and the process is re-run. When all parties are satisfied with the service, which may require several iterations around the refine/simulate loop, the code generator is used to produce the C++ code which is then compiled and executed on the target speech applications platform.

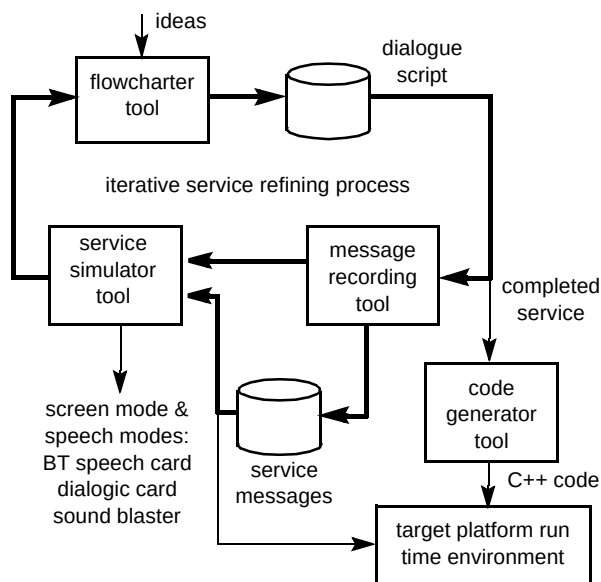


Fig 1 VISAGE service-design process.

Figure 2 shows the VISAGE architecture that has been developed to support this process.

The building block library is the foundation upon which the rest of the system is built and is sole repository for all building block information. For the flowcharter it contains the design-time building blocks used for designing services and the corresponding help information. For the simulator and interpreter environment, it contains all information pertinent to the run-time behaviour of the building blocks. By having a single repository of building block information held entirely separately from the VISAGE tool-set, new

building blocks may be added to the tool-set library without modifying the tool-set in any way.

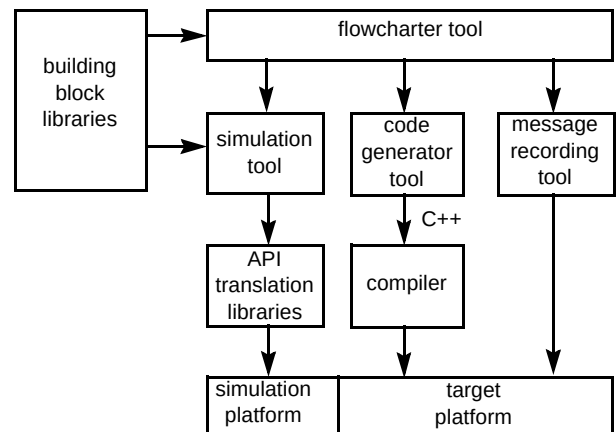


Fig 2 VISAGE architecture.

The essence of this new architecture is to take advantage of object oriented (OO) design techniques. OO design has become popular in recent years as software engineering becomes a more mature industry with better design and development techniques. With OO, systems are decomposed based on natural partitioning of the problem domain — this is termed 'real world modelling'. It is easier to understand, and therefore maintain, because it closely represents the real world, and the designs produced are more durable and flexible. This allows the reuse of component software within the system, without the need for rebuilding and consequently re-testing the whole system if only a single component changes.

VISAGE has been modelled as classes that could be reused across the tool-set. These classes have been grouped into categories of related functionality. This can be useful if only a sub-set of the available functionality is required. For instance, an application not related to service creation tools at all, could use just the 'general' category which contains classes for strings and lists. Alternatively it could use all the categories except 'media' if no hardware references were required. An example of this could be a design tool, which is only concerned with representing a service in a graphical format, not in the execution of the service.

The VISAGE class category is shown in Fig 3. The arrows show a 'makes use of' relationship, e.g. 'dialogue' makes use of components in 'general'.

The 'blocks' category is the first major point of flexibility, since this is likely to be the area where the most frequent changes are to be made. A common block model exists, which is implemented as a dynamically linkable library (DLL). Their design time properties (i.e. the block parameters and connections to other blocks) are used by the flowcharter, and their run-time properties (i.e. the behaviour

of the block) are used by the simulator and the code generator.

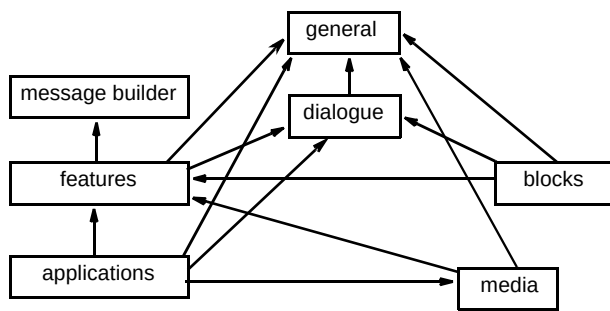


Fig 3 VISAGE class categories.

The 'media' category gives VISAGE independence from the target platform by having a common class model to represent the service execution aspects of a 'generic speech platform'. This assumes that all platforms will have a similar base set of capabilities for running interactive speech services, e.g. recording and play-back of messages, speech recognition and DTMF detection. Different implementations of this class would exist for each target platform, but because they are all based on a common model the interface to them is the same. So a service can be ported from one platform to another simply by plugging in a new media, without the need to re-test the whole service.

Within VISAGE a voice interactive service is termed a 'dialogue'. The 'dialogue' category contains the classes responsible for the representation of a service. The service is represented as blocks of functionality, linked together with connectors which can have conditions associated with them. Blocks can be grouped together into 'pages', which are analogous to subroutines in a programming language. A parser exists to read in the dialogue script, described above, into the dialogue category. Changes in the script language can be catered for by changing this parser, and leaving the dialogue category the same. This means that client software of the dialogue category would remain unchanged.

The 'features' category contains classes for the platform-independent modelling of dialogue objects such as recorded messages, vocabularies, etc.

The 'message builder' category contains classes for numeral information, in order to build up messages containing numbers, telephone numbers, dates and money in specific formats.

Each of the categories described above has been implemented as separate dynamically linkable libraries (DLLs). This has the advantage of allowing updates of the software to be issued in a minimal number of files, and allowing a sub-set of the functionality to be used. Also, each building block has been implemented as a separate DLL;

this allows parties other than the developers to extend the set of building blocks without needing to re-build the tool set.

4.2 Flowcharter

To create the service logic, building blocks taken from a palette of generic building blocks are selected and placed on the flowchart canvas. An example flowchart is shown in Fig 4. These are linked together to form the logical flow of a voice interactive service. The names of the links define the condition under which that link is taken. The building block range consists of:

- telephony — call answer, call hang-up, dial out, call transfer block, examine line parameters and several call control blocks,
- dialogue — information play, DTMF (TouchTone) input, speech recognition, 'yes/no' speech recognition, record-message and play-message blocks,
- non-dialogue — stub block for access to C++ external functions (to external DOS and UNIX programs, for example, with a database to retrieve information) switch on condition, test condition, utility arithmetic and logging blocks,
- miscellaneous — page blocks used for service decomposition. These are analogous to conventional programming language subroutines.

A parameter form is associated with each building block and is used to specify message prompts and parameters. An example DTMF input parameter form is shown in Fig 5. This form specifies a service prompt, and then expects a DTMF key between 0 and 9 to be pressed as a response. The three prompts contain progressively more explicit directions to the user. If the first prompt does not receive the correct response, the second is played. If this still does not receive the correct response, the third is played. If this also fails, the whole block is deemed to have failed and appropriate action is taken, e.g. transfer the call to an operator. The message prompts associated with all dialogue building blocks may be specified for two modes of dialogue operation, 'normal' and 'fast track'. The latter message prompts are usually more concise and played out only for experienced callers. Help messages may also be specified. The dialogue building blocks that require a response from the caller (e.g. speech recognition blocks) may also specify messages to be played if an unexpected response is received. These can be a mumble (unrecognised speech), silence, block fail (e.g. three incorrect responses) and a system fail (e.g. a failure in the application platform). Arithmetic operations may also be specified on each form to enable, typically, service loops

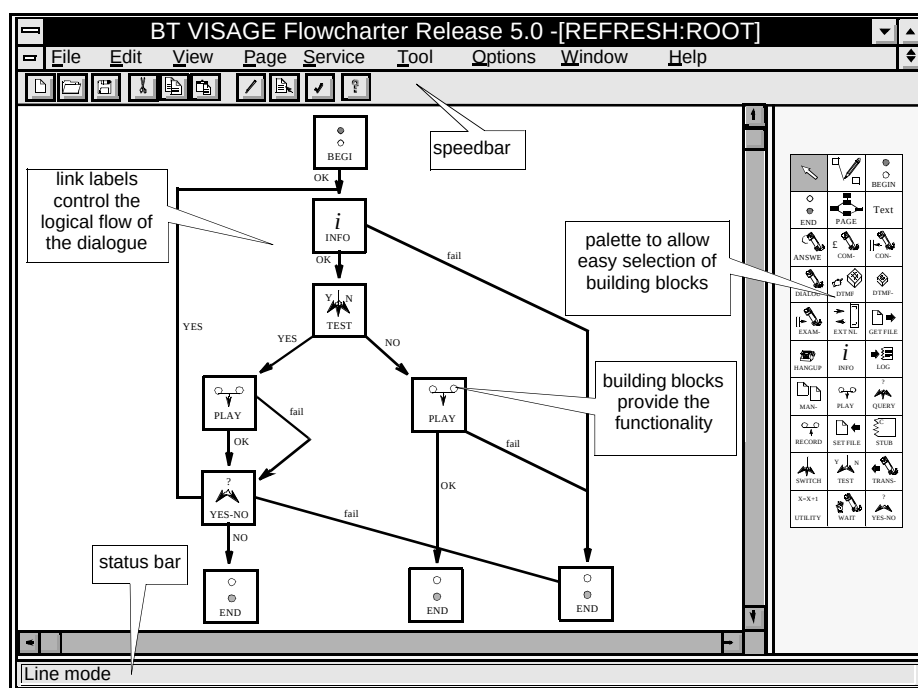


Fig 4 VISAGE flowchart canvas.

and counts to be implemented, e.g. to allow the user to run through the service three times before they have to hang up.

Parameters used across the whole service may be specified globally, for example, variables, constants, global messages common to a range of blocks and service specific parameters, such as the line signalling system used.

To facilitate the development of large services, pages may be used to expand the flowchart canvas in a similar fashion to using conventional programming subroutines. Likewise parameters may be specified to pass between pages.

Services may be validated to ensure that the service logic and parameters comply with predefined service constraints and documented automatically as part of the service creation tool. The documentation consists of a range of top-level flowcharts to convey to clients the basic functionality and low-level documentation to enable software developers to modify and expand the application efficiently. Finally a script describing the service is produced for message recording, simulation and code generation purposes.

The flowcharting tools have been developed using cross-platform development software to enable the flowcharting software tool to be compiled for execution on a PC using Windows, and on UNIX platforms (Sun and HP) using the motif windowing environment.

4.3 Message recording

Message recording uses an automatic batch loading facility which permits the complete set of service messages to be displayed, recorded, edited and loaded into appropriate speech files without any specialised knowledge. Two main message recording and checking options are currently available:

- record missing messages — this option checks and prompts for missing messages and for newly created services, enabling the user to record all messages for that service,
- review messages — this option permits the user to review all service messages and to discard unwanted ones.

The message recording tool uses Microsoft's media control interface (MCI) API. This allows the message recording tool to be independent of the underlying hardware. The current preferred product is Sound Blaster, but any media device with an MCI interface can be used.

The messages are stored in wave audio file format. This allows the messages to be edited using a commercially available editing tool. Utilities exist for converting wave files into those suitable for the BT speech card [4] and BT's interactive speech applications platform (ISAP) [3].

DTMF BLOCK

*Block Name:

Messages

☒ Normal ☐ Fast-Track

*Prompt 1:

Prompt 2:

Prompt 3:

☐ Invalid key:

☐ No key pressed:

☐ Block fail:

☐ System error:

Parameters

☒ Flush key-ahead buffer

Digit string length: Max no. of digits:

☒ Variable ☐ Fixed

☒ Silence timeout: default user def (secs)

☒ Final timeout:

DTMF key(s) received:

Pre-and Post-block Operations

```

graph TD
    Pre-stub --> Pre-Arith
    Pre-Arith --> This block
    This block --> Post-Arith
    Post-Arith --> Post-stub
  
```

Ok Cancel Help

Fig 5 VISAGE DTMF parameter form.

4.4 Service simulation

Service simulation is an important aspect of service creation and permits a very rapid testing/modification cycle, off-line and without the need for code compilation and downloading on to the target speech platform. It is also particularly useful for service demonstration without the speech hardware platform in place. The service simulator may be launched from within the flowcharter, allowing the simulator to run the service currently being developed. The service simulator operates in a number of modes.

- **VDU mode** — simulation in this mode is used prior to message prompt recording. Service prompts are displayed as text and user responses simulated via keyboard interaction. This allows the simulation of all possible paths through the service, including error conditions. Error conditions, like a mumbled response, can easily be generated in a simulated environment and a log produced of the tests performed in order that they can be repeated.
- **Telephone mode** — a service may be simulated as it would appear to a real caller. The simulator can use a BT speech card, Dialogic's voice processing board or Sound Blaster card, for all telephony functions that the interfaces support. In the case of the BT speech card, this includes playing a message, DTMF and speech recognition, and recording and playing back caller messages. There are two modes of operation.

— **Simulated recognition** — this uses a 'Wizard of Oz' approach, where an operator simulates the speech recogniser via keyboard interaction. The caller is completely unaware of this — they think they are talking to a machine which recognises their speech. This approach allows services employing advanced recognition to be simulated prior to template collection or the availability of a suitable recogniser. The recognition success may be pre-defined to simulate, for example, 98.5% recognition success, or alternatively may simulate the actual performance of a specified recogniser.

— **Automatic recognition** — in this mode the simulator uses the speech recogniser of the BT speechcard. Templates exist for speaker-independent recognition of various sets of phrases that one would expect during a normal speech dialogue, for example 'yes/no' and the digits 0—9.

The script from the flowcharter, containing the logical flow and the block parameters, is first parsed to create a dialogue object based on the 'dialogue' class described above. Functions within that class are used to perform the parsing.

At this stage there exists in memory a representation of the service containing pages, blocks, interconnecting block links and all data used within the service.

Executing the service on a specific platform (e.g. the BT speech card) involves telling the dialogue to execute itself on that specified media. The pages, consisting of blocks, execute themselves and in turn the blocks execute. The building block run-time behaviour is not stored in the dialogue itself, but calls on each of the blocks' DLLs to perform the block execution.

During simulation, break points can be added and service data (variables) examined to help correction of defects in a service.

A simplified simulator has also been developed for use as a marketing tool. This runs on a PC (typically a portable PC) using a Sound Blaster speech interface for playing and recording messages. DTMF interaction is simulated via a graphical representation of a telephone and voice recognition simulated using screen interaction. In conjunction with the flowcharting and message-recording tools, this suite of tools is particularly useful for demonstration purposes with a view to being used for requirements capture and specification of new services.

4.5 Code generator

The current code generator runs on a Sun SparcStation and takes script produced by the VISAGE flowcharter tool and generates C++ code specifically for ISAP. The script contains the block parameter information and the logical flow of the dialogue. The behavioural information of the blocks is stored in the code generator. The code produced is then compiled and run on the target application platform. A variety of reports are produced that provide service details, logical and physical file name associations and all variables and arguments passed between program layers. A 'makefile' is also produced to facilitate code compilation on a UNIX machine.

Use of the classes described above is being considered to produce a more flexible and maintainable solution to producing code that can execute on the BT ISAP. This would make a memory model of the service script, in the same manner as the simulator, producing a dialogue object. This dialogue object can then be executed on the specified media, in this case the BT ISAP. The execution of each block within the service uses the building block DLLs, in an identical manner to the simulator, although these would be re-compiled for the specific target environment.

This approach has the advantage in that much of the code is reused from other parts of the VISAGE tool set, greatly reducing the time to test this tool. Equally important is the way blocks are held as separate pluggable components. This has two distinct advantages:

- if a new block is defined for a specific service or an updated version of a block is produced, this may be added to the system without the need for re-building the main executable part of the service — this is important for services already deployed in the field,
- services can share the same building block executable code, saving memory space.

At the time of writing, a BT ISAP media object has been produced and tests are under way to check the viability of using this method.

5. VISAGE uses to date

VISAGE has been used in the development of a range of services, including 'CallMinder', the network-based call answering service, and the automated customer interface into BT's customer service system for ordering network service. A new service based on BT charge cards, called cashless store and forward, also used VISAGE to develop its dialogue. This service allows somebody using a charge card to make a call, and then, if they cannot get through, because, for example, the line is busy, they can store their message, for which delivery attempts will then be made at regular intervals.

A number of new services currently being trialled have also used VISAGE in their development. These include telemetry MeterLink [4], a system where domestic meters are read overnight using the telephone, automation of directory enquiry services [5], where customers can get a phone number from the directory without having to speak to a human operator, and voice-controlled dialling and transaction processing services.

6. Conclusions

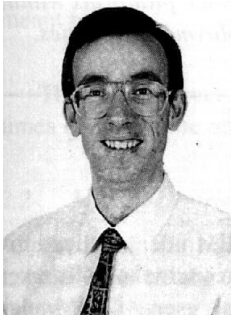
This paper has discussed the benefits and functionality of service creation tools. A BT platform-independent tool, VISAGE, has also been described that is currently used for the creation and maintenance of advanced network-based services. Using the tool a plethora of benefits have been demonstrated and valuable feedback gained from service creation developers to drive ongoing tool developments. In addition, VISAGE is being used as a portable and effective marketing tool, for demonstrating voice interactive services with a view to new service generation.

Acknowledgements

The authors wish to acknowledge the contribution made by colleagues in the VISAGE service creation team.

References

- 1 Beacham K and Barrington S: 'CallMinder — the development of BT's new telephone answering service', BT Technol J, 14, No 2, pp 52—59 (April 1996).
- 2 BT Publication: 'Now you're talking ... voice services', Ref No 367:TEC4:REPO/8 (1994).
- 3 Rose K R and Hughes P M: 'The BT speech applications platform', British Telecommunications Eng J, 13, pp 304—310 (January 1995).
- 4 Welsby K et al: 'MeterLink™ — BT's new telemetry service for meter reading', BT Technol J, 14, No 2, pp 60—68 (April 1996).
- 5 Orrey D A and Adams LR: 'Operator assistance voice automation trials', BT Technol J, 14, No 2, pp 69—74 (April 1996).



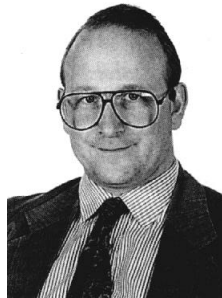
Tony Chester joined BT in 1987 after receiving a BSc in electronic engineering. He initially worked on data communications equipment, writing control software for modems and multiplexers.

Since then he has worked in facilities management, where BT manage companies' telecommunications infrastructure, and video conferencing on BT's conference room cabinet, the VC5000. He joined the VISAGE team in 1995.



Roger Hanes joined BT in 1975 after receiving a BTech and PhD degree in telecommunications. He has worked on nano-second digital multiplex equipment, trans-multiplexer theoretical studies, low bit rate speech encoding and, more recently, the development of service creation tools for the ISAP and keeping a watching brief on competitor developments.

He is currently the tools development team leader of the VISAGE service creation team.



John Cheshire started work at BT in 1985 following several years as a silicon chip designer at Racal Microelectronic Systems Ltd. He initially worked as part of the ASTRA VLSI design team and moved into speech applications in 1987.

Since then he has managed a number of successful developments including the VISAGE service creation project. He also managed a number of leadership events and helped to provide steer to the network operations breakout theme.

He has been extensively involved with supplier evaluation, setting up and managing a number of outsourced software packages.